

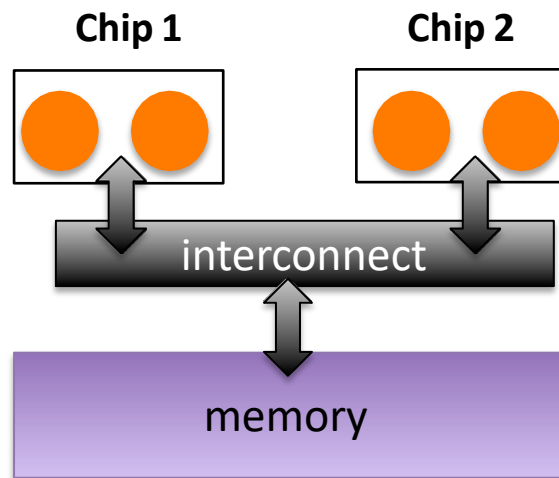
Lecture 6:

Shared--Memory Programming with OpenMP

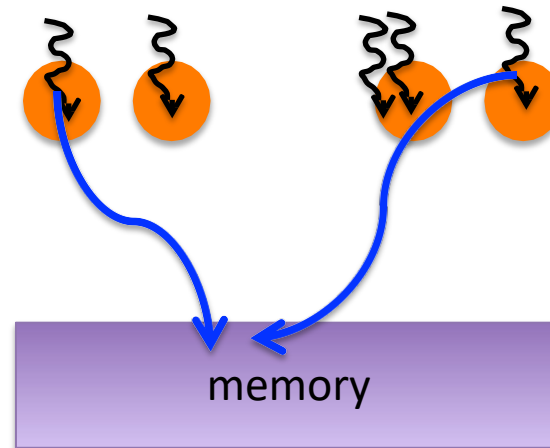
Beakal Gizachew Assefa

Shared--Memory Programming Model

- More correct name: Shared--address space programming
 - Threads communicate through shared memory as opposed to messages
 - Threads coordinate through synchronization (also through shared memory).



Recall shared memory system
(can be either UMA, NUMA)

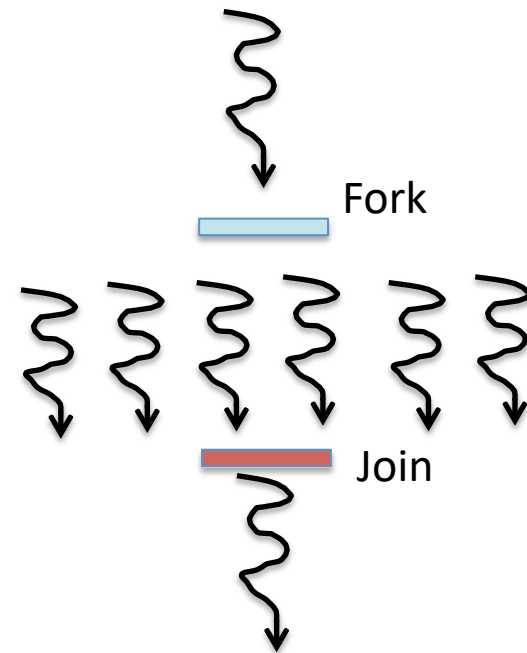


OpenMP

- Chapter 5 from the textbook
- Tutorial: <https://computing.llnl.gov/tutorials/openMP/>
- Standardization Committee: <http://www.openmp.org>
- Model for shared-memory parallel programming
 - Prevailing approach in scientific computing community
 - A simplified alternative to Pthreads
- OpenMP
 - MP= multiprocessing
 - Open= open specification, developed by community
- Extensions to existing programming languages (Fortran, C and C++)
 - mainly by directives/pragmas
 - a few library routines

OpenMP Execution Model

- Fork-join model of parallel execution
 - Begin execution as a single process (**master thread**)
- Start of a parallel construct:
 - Master thread creates team of threads (**worker threads**)
- Completion of a parallel construct:
 - Threads in the team synchronize – **implicit barrier**
- Only master thread continues execution



OpenMP uses #pragmas

- Programmer annotates the code with pragmas (directives)
 - Pragmas are special preprocessor instructions.
- Compilers that don't support the **pragmas ignore them.**
- The interpretation of OpenMP pragmas
 - They modify the statement immediately following the pragma
 - This could be a compound statement such as a loop

#pragma omp ...

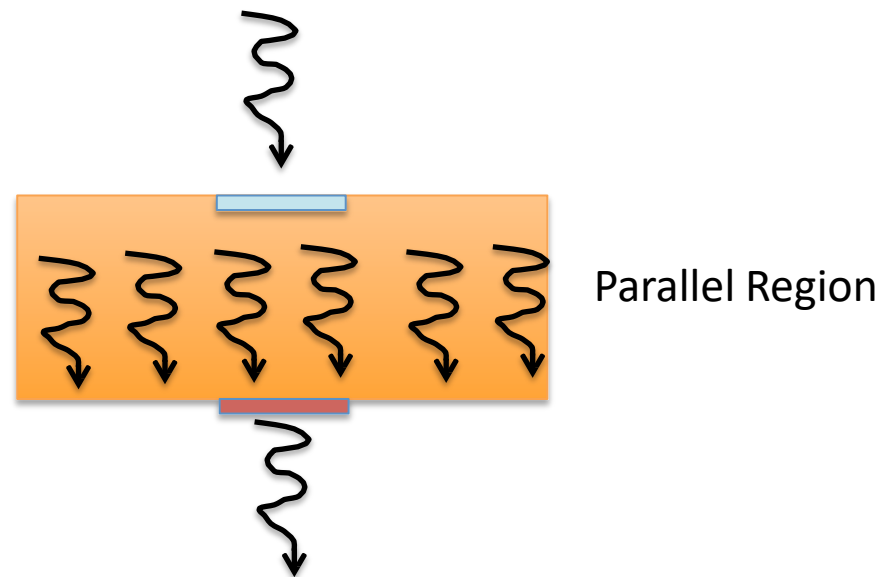
OpenMP Constructs

- Today
 - Parallel Region
 - Parallel For loop
 - Critical Directive
 - Reduction

OpenMP Parallel Region

```
#pragma omp parallel [ clause [ clause ] ... ] new-line  
{  
    //structured block  
}
```

- Block of code to be executed by multiple threads in parallel
- Each thread executes the **same code redundantly (SPMD)**



Hello World-Serial

```
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char* argv[]){

    int thread_count= strtol(argv[1], NULL, 10)

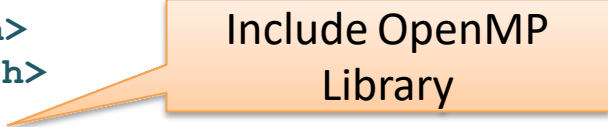
    int myID = 0;

    printf("Hello from thread %d of %d\n", myID, thread_count);

    return 0;
}
```


Hello World in OpenMP

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```



Include OpenMP
Library

```
int main (int argc, char* argv[]){
    int thread_count= strtol(argv[1], NULL, 10)

    int myID = 0;

    printf("Hello from thread %d of %d\n", myID, thread_count);

    return 0;
}
```

Hello World in OpenMP

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

Include OpenMP
Library

```
int main (int argc, char* argv[]){
```

```
    int thread_count= strtol(argv[1], NULL, 10)
```

```
    int myID = omp_get_thread_num();
```

Get thread ID

```
    printf("Hello from thread %d of %d\n", myID, thread_count);
```

```
    return 0;
```

```
}
```

Thread IDs start from 0 and goes to number of threads -1

Hello World in OpenMP

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
int main (int argc, char* argv[]){
    int thread_count= strtol(argv[1], NULL, 10);
```

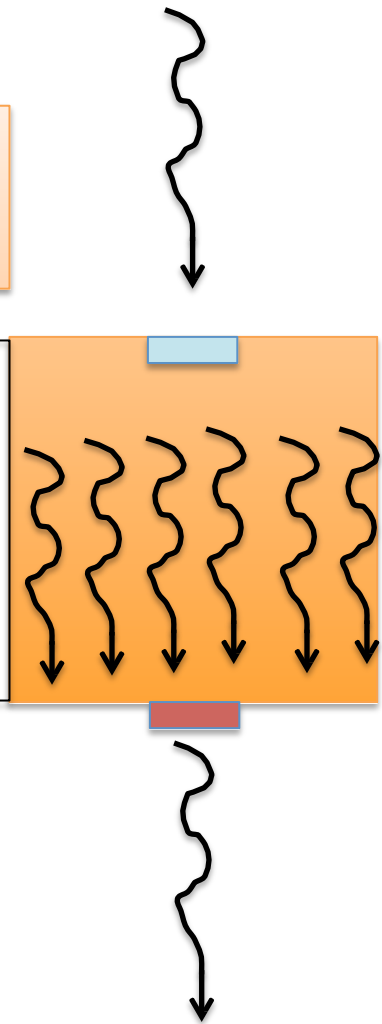
Specify number
of threads to
create

```
#pragma omp parallel thread_num(thread_count)
{
    int myID = omp_get_thread_num();

    printf("Hello from thread %d of %d\n", myID, thread_count);
}
```

```
return 0;
```

```
}
```



Hello World in OpenMP

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main (int argc, char* argv[]){

    int thread_count= strtol(argv[1], NULL, 10)

    #pragma omp parallel thread_num(thr
    {
        int myID = omp_get_thread_num();

        int thread_count = omp_get_num_threads();
        printf("Hello from thread %d of %d\n", myID, thread_count);
    }

    return 0;
}
```

How many
threads actually
created?

The OpenMP standard doesn't guarantee that *num_threads* clause will actually start *thread_count* threads.

OpenMP Runtime Library

- In addition to compiler directives/pragmas, OpenMP has a runtime

```
int omp_get_num_threads(void);
```

- Returns the number of threads currently in the team executing the parallel region from which it is called

```
int omp_get_thread_num(void);
```

- Returns the thread number, within the team, that lies between 0 and `omp_get_num_threads()-1`, inclusive. The master thread of the team is thread 0

Hello World in OpenMP

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
int main (int argc, char* argv[]) {
```

```
    int thread_count= strtol(argv[1], NULL, 10)
```

```
    int myID = omp_get_thread_num();
```

```
    #pragma omp parallel thread_num(thread_count)
    {
```

```
        int thread_count = omp_get_num_threads();
```

```
        printf("Hello from thread %d of %d\n", myID, thread_count);
```

```
    }
```

```
    return 0;
```

```
}
```

Calling this outside of a
parallel region will get us 0.

Compiling

```
gcc -g -Wall -fopenmp -o omp_hello omp_hello.c
```

```
./omp_hello 4
```

running with 4 threads

compiling

Hello from thread 0 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 3 of 4

possible
outcomes

Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4
Hello from thread 3 of 4

Hello from thread 3 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4

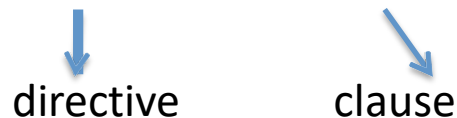
Hello World in OpenMP

- Things to think about
 - Code should be correct without the pragmas and library calls
- Differences from Pthreads
 - More of the required code is managed by the compiler and runtime (The code is shorter)
 - There is an implicit thread identifier
 - Thread creation and termination (fork and join) are implicit
 - Barrier at the end of a parallel region is implicit

OpenMP Clauses

- Text that modifies a directive/pragma/construct.
 - For example, the **num_threads** clause can be added to a parallel directive.
 - It allows the programmer to specify the number of threads that should execute the following block.

pragma omp parallel num_threads (thread_count)


directive clause

- There are many more clauses – will learn most of them soon

Parallel Loop Construct

- Compiler calculates loop bounds for each thread directly from *serial* source (computation decomposition)
- Compiler also manages data partitioning
- Implicit barrier at the end of the loop
- This assumes a parallel region has already been initiated, otherwise it executes in serial on a single processor.

Serial Program	Parallel Program	Parallel Program
<pre>double res[N]; for(int i=0; i < N; i++) do_huge_comp(res[i]);</pre>	<pre>double res[N]; #pragma omp parallel { #pragma omp for for(int i=0; i < N; i++) do_huge_comp(res[i]); }</pre>	<pre>double res[N]; #pragma omp parallel for for(int i=0; i < N; i++) do_huge_comp(res[i]);</pre>

Limitations and Semantics of Parallel for

- Requirements ensure iteration count is predictable
- Not all loops can be ||ized
 - Loop index: signed integer
 - Program correctness cannot depend on which thread executes a particular iteration
 - Loop control parameters must be the same for all threads
 - Termination Test: <, <=, >, >= with loop **invariant** integer
 - Incr/Decr by loop invariant int; change each iteration
 - Count up for <, <=; count down for >, >=
 - Illegal to branch out of a loop

```
#pragma omp parallel for  
for(int i=lbound; i < ubound; i++)  
    do_huge_comp(res[i]);
```

Data Sharing

- Shared memory parallel programs often employ two types of data
 - Shared data, visible to all threads
 - Private data, visible to a single thread (often stack--allocated)
- PThreads:
 - Global--scoped variables are shared
 - Stack--allocated variables are private
- OpenMP:
 - **shared** variables are shared
 - **private** variables are private
 - Default is **shared**
 - Loop index is **private**

Data Sharing

- PThreads:

- Global--scoped variables are shared
- Stack--allocated variables are private

```
// shared, globals
int bigdata[1024];
//thread function
void* foo(void* bar) {
    // private, stack
    int tid;

    /* Calculation goes
       here */
}
```

- OpenMP:

- **shared** variables are shared
- **private** variables are private
- Default is **shared**
- Loop index is **private**

```
int bigdata[1024];
void* foo(void* bar) {
    int tid;
    #pragma omp parallel
        shared ( bigdata ) \
        private ( tid )
    {
        /* Calc. here */
    }
}
```

OpenMP Critical Directive

- Enclosed code executed by all threads, but
 - **restricted to only one thread at a time**

```
#pragma omp critical [ ( name ) ] new-line  
structured--block
```

- A thread waits at the beginning of a critical region until no other thread in the team is executing a critical region with the same name.
- All unnamed critical directives map to the same unspecified name.
- Acts like a mutex/lock

Recall our parallel sum example

- Recall our parallel sum example from Lecture 4

```
int items_per_task = n/t;
mutex m;
int my_sum, my_x, sum;
int start = thread_id * items_per_task;

for (i=start; i<start + items_per_task; i++) {
    my_x = Compute_next_value(...);
    my_sum += my_x;
}
mutex_lock(m);
sum+= my_sum;
mutex_unlock(m);
```

OpenMP version of sum example

- Replaced mutex with a critical section

```
int items_per_task = n/t;  
mutex m;  
int my_sum, my_x, sum;  
int start = thread_id * items_per_task;  
#pragma omp parallel for  
for (i=0; i< N; i++) {  
    my_x = Compute_next_value(...);  
    my_sum += my_x;  
}  
#pragma omp critical  
sum += my_sum;
```

Is this correct?

No, my_x and my_sum should be private, otherwise race condition!

OpenMP version of sum example

- Specify thread--private variables

```
int items_per_task = n/t;  
mutex m;  
int my_sum, my_x, sum;  
int start = thread_id * items_per_task;  
#pragma omp parallel for private(my_x, my_sum)  
for (i=0; i< N; i++) {  
    my_x = Compute_next_value(...);  
    my_sum += my_x;  
}  
#pragma omp critical  
sum += my_sum;
```

Is this correct?

No, critical section should be in a parallel region, otherwise only master thread executes that section

OpenMP version of sum example

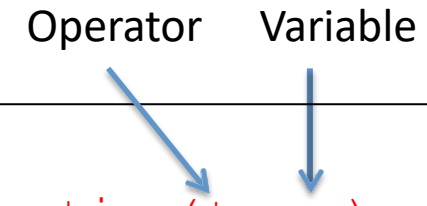
- Critical section should be in a parallel region

```
int items_per_task = n/t;  
mutex m;  
int my_sum, my_x, sum;  
int start = thread_id * items_per_task;  
#pragma omp parallel private(my_x, my_sum) shared(sum)  
{  
    #pragma omp for  
    for (i=0; i< N; i++) {  
        my_x = Compute_next_value(...);  
        my_sum += my_x;  
    }  
    #pragma omp critical  
    sum += my_sum;  
}
```

OpenMP Reduce

- OpenMP has reduce operation

```
sum = 0;
#pragma omp parallel for private(my_x) reduction(+:sum)
for (i=0; i < 100; i++) {
    my_x = Compute_next_value(...);
    sum += my_x;
}
```



- Reduce ops and init() values (C and C++):

+	0	bitwise &	0	logical &	1
-	0	bitwise	0	logical	0
*	1	bitwise ^	0		

FORTRAN also supports min and max reductions

OpenMP Reduction

- OpenMP runtime/compiler has an “efficient” implementation for parallel reduction

```
int sum;  
#pragma omp parallel for reduction(+:sum)  
    for (i=0; i< N; i++)    {  
        sum += array[i];  
    }
```

OpenMP is a collection of

- Pragma
 - Annotates the program to indicate parallel regions etc.
 - Interpreted by compiler
- Runtime library calls
 - E.g. `omp_get_thread_num()`
 - `<omp.h>`
- Environment variables
 - `OMP_NUM_THREADS`

Environment Variables

OMP_NUM_THREADS

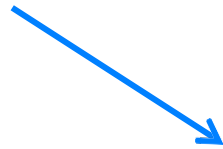
- Sets the number of threads to use during execution
- When dynamic adjustment of the number of threads is enabled, the value of this environment variable is the maximum number of threads to use
- If undefined one thread per CPU is used.
- For example,
`setenv OMP_NUM_THREADS 16 [csh, tcsh]`
`export OMP_NUM_THREADS=16 [sh, ksh, bash]`
- When a job is submitted through a scheduler, you can set the number of threads in the job script

There are other environment variables for OpenMP – will learn them later

No Compiler Support?

- In case the compiler doesn't support OpenMP

`# include <omp.h>`



`#ifdef _OPENMP`

`# include <omp.h>`

`#endif`

No support?

```
# ifdef _OPENMP
```

```
    int my_rank = omp_get_thread_num ( );
```

```
    int thread_count = omp_get_num_threads ( );
```

```
# e l s e
```

```
    int my_rank = 0;
```

```
    int thread_count = 1;
```

```
# endif
```


OpenMP Compilers

- GNU Compiler (gcc, gfortran, g++)
 - `-fopenmp`
- Intel
 - `-openmp`
- PGI (Portland Group Compilers)
 - `-mp`
 - Acquired by Nvidia
- Cray, IBM and other compilers support OpenMP
- Version of the compiler may make a difference
 - New clauses may not be supported by all the compilers

A Programmer's View of OpenMP

- OpenMP will:
 - Allow a programmer to separate a program into *serial regions* and *parallel regions*, rather than concurrently-executing threads.
 - Hide stack management
 - Provide synchronization constructs
- OpenMP will not:
 - Parallelize automatically
 - Guarantee speedup
 - Provide freedom from data races

Final Remarks

- Incremental parallelization
 - Parallelize individual computations in a program while leaving the rest of the program sequential
- Compiler based
 - Compiler generates thread program and synchronization
- System is viewed as collection of cores all of which have access to a shared main memory
- OpenMP1.0 for Fortran in 1997 and for C/C++ in 1998
- OpenMP4.0 released in 2013

Acknowledgments

- These slides are inspired and partly adapted from
 - Didem Unat (Koç University)
 - Mary Hall (Univ. of Utah)
 - The course book (Pacheco)